KAPITEL 9: REST

LERNZIELE

- Die fünf Grundprinzipien von REST erläutern können.
- Einfache REST-Schnittstellen entwerfen und mit PHP umsetzen können.
- Die Vor- und Nachteile von REST erklären können.

9.1 ÜBERBLICK

- REST steht für Representational State Transfer und geht zurück auf die Dissertation von Roy Fielding aus dem Jahr 2000.
- REST ist ein Architekturstil für Schnittstellen von Webanwendungen mit folgenden Grundprinzipien:
    - Eindeutige Identifikation von Ressourcen (=Fachobjekte) über URIs
    - Ressourcen-Verknüpfung: Ressourcen sind mit anderen Ressourcen über URIs verknüpft (z.B. Bestellung hat Kunde)
    - HTTP-Standardmethoden: Server werden über HTTP mit den Grundbefehlen GET, PUT, POST, DELETE aufgerufen
    - Unterschiedliche Repräsentationen: Zum Datenaustausch können unterschiedliche Repräsentationen (Text, XML, JSON, HTML) genutzt werden
    - Statuslose Kommunikation: Die Kommunikation mit dem HTTP-Server ist zustandslos (stateless http server), der Server ist damit sehr gut an steigende Nutzerzahlen anpassbar (Skalierbarkeit).



EINDEUTIGE IDENTIFIKATION VON RESSOURCEN

Jede Ressource wird eindeutig über einen URI identifiziert.



Beispiele für Ressourcen:

URI	Interpretation
http://example.com/customers/1234	Kunde mit ID 1234
http://example.com/orders/2007/10/776654	Bestellung vom Oktober 2007 mit ID 776654
http://example.com/products/4554	Produkt mit ID 4554
http://example.com/processes/sal-increase-234	Prozess Gehaltserhöhung für Quartal 2, 3 und 4

Anmerkungen:

- Der Fokus liegt auf den Ressourcen (=Fachobjekten). Zum Beispiel:
  - Streaming-Dienst: Film, Serie, Episode, Benutzer, Watchlist, Genre, Bewertung, Streaming-Lizenz, Rating
  - Social Media: Benutzer, Post, Kommentar, Gruppe, Event, Freundschaft, Nachricht, Hashtag, Story, Feed
  - Reiseportal: Hotel, Flug, Buchung, Reisender, Bewertung, Reiseziel, Verfügbarkeit, Preiskategorie, Route, Versicherung
  - Banking-App: Konto, Transaktion, Kunde, Kreditkarte, Dauerauftrag, Anlageprodukt, Kontoauszug, Überweisung, Standing Order, Investment
- Eine Webanwendung, die einen REST-Server für ihre Datenversorgung nutzt, ist entsprechend "Daten-lastig".
- Für Anwendungen, in denen Abläufe bzw. Prozesse oder Algorithmen stark im Vordergrund stehen, ist dieses Paradigma nicht ideal

RESSOURCEN-VERKNÜPFUNG

REST ist ressourcenorientiert. Die mit REST bereitgestellten Ressourcen entstammen einem Datenmodell und können über Anfragen abgefragt, verändert, gelöscht oder hinzugefügt werden..

- Datenmodelle aus Objekten mit Referenzen auf andere Objekte werden übersetzt in Ressourcen-Netze.
- Eine Ressource kann beliebig viele Verknüpfungen (Links) zu anderen Ressourcen besitzen.

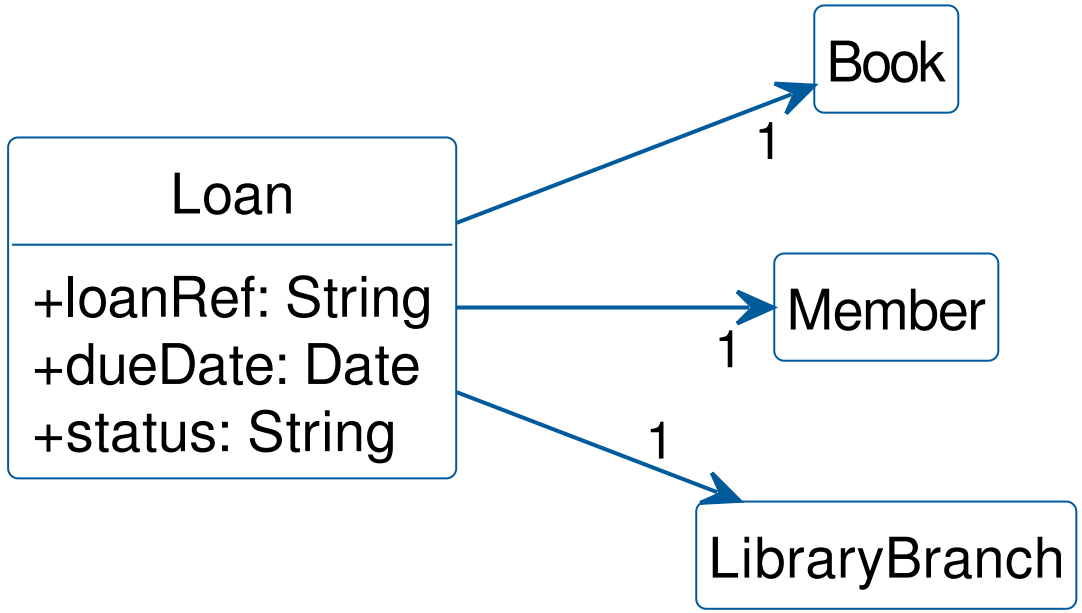


Gegeben sei eine Ausleihe in einem Bibliothekssystem. Diese hat eine Fälligkeit, Status und ID, sowie die referenzierte Ressourcen Buch, Mitglied und Bibliothek.

Beispiel für eine solche Ressourcenverknüpfung in JSON

```
{
    "loanRef": "http://library.com/loans/78901",
    "bookRef": "http://library.com/books/5678",
    "memberRef": "http://library.com/members/9012",
    "libraryBranchRef": "http://library.com/branches/345",
    "dueDate": "2024-04-15",
    "status": "active"
}
```

Das resultierende Klassenmodell könnte dabei wie folgt aussehen:



Dies enthält die Klasse des gezeigten Objektes (Loan) sowie alle referenzierten Klassen und die bekannten Attribute in Loan. Die REST-Schnittstelle liefert uns die Möglichkeit mit diesen Ressourcen zu arbeiten. Über die Schnittstelle kann mit Loan, Book, Member und LibraryBranch-Objekten gearbeitet werden.

ÜBUNG: RESSOURCEN-VERKNÜPFUNG

Gegeben sei folgendes Order-Objekt (orderRef ist die ID).

```
{
    "orderRef":     "http://example.com/orders/1234",
    "amount":       23,
    "productRef":   "http://example.com/products/44533",
    "customerRef":   "http://example.com/customers/4711"
}
```

Wie sieht das zugehörige UML-Klassenmodell aus? Benennen Sie kurz die entsprechenden Klassen.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

STANDARDMETHODEN

Operationen eines REST-Service beziehen sich auf eine Ressource und können mit den Mitteln, die HTTP bietet, aufgerufen werden.

Die typischen HTTP-Methoden GET, POST, PUT und DELETE haben in diesem Zusammenhang eine Standard-Interpretation:

Methode	Interpretation
GET	hole Information zu ein oder mehreren Ressource
POST	erzeuge Ressource zu übergebenen Werten
PUT	aktualisiere Ressource bzw. erzeuge Sub-Ressource oder füge Sie an
DELETE	Löschen einer Ressource

Beispiele für Operationen:

```
GET http://localhost/restService/bestellungen
GET http://localhost/restService/bestellung/1
POST http://localhost/restService/bestellung
PUT http://localhost/restService/bestellung/1
DELETE http://localhost/restService/bestellung/1
```

- Liefere alle Bestellungen
- Liefere die Bestellung mit der ID 1
- Erzeuge Bestellung zu im http-body enthaltenen Daten (z.B. als JSON-Objekt)
- Aktualisiere die Bestellung mit der ID 1 aus den im http-body enthaltenen Daten
- Lösche die Bestellung mit der ID 1



Wie in HTTP definiert, ergeben sich für die verwendeten HTTP-Methoden Anforderungen, ob ein Payload mitgesendet werden muss oder nicht, sowie ob eine entsprechende Antwort in einem bestimmten Format entsteht.

Zum Beispiel liefert GET entweder ein einzelnes Element der Ressourcenklasse oder eine Menge. POST dagegen sendet ein Objekt, PUT Teile des Objektes.



Die Loan-Ressource aufgreifend könnte man folgende Tabelle ableiten:

HTTP-Methode	URL	Request Payload	Response Payload	Beschr
GET	`http://library.com/loans/78901`	-	Loan	Liest ei spezifis Ausleih
GET	`http://library.com/loans`	-	Loan[]	Liest al Ausleih
POST	`http://library.com/loans`	Loan	Loan	Erstellt neue Au
PUT	`http://library.com/loans/78901`	Loan	Loan	Aktuali eine besteh Ausleih
DELETE	`http://library.com/loans/78901`	-	-	Löscht Ausleih



Anmerkungen:

- Bei GET und DELETE wird kein Request Payload benötigt
- DELETE liefert typischerweise keinen Response Payload
- Bei POST wird die ID/Ref meist vom Server generiert
- Bei PUT werden nur die zu ändernden Felder übermittelt
- Die Response enthält üblicherweise die komplette, aktualisierte Ressource



Die Book-Ressource dagegen könnte wie folgt definiert werden:

HTTP-Methode	URL	Request Payload	Response Payload	Beschr
GET	`http://library.com/books/78901`	-	Book	Liest ei spezifis Buch
GET	`http://library.com/books`	-	Book[]	Liest al Bücher
POST	`http://library.com/books`	Book	Book	Erstellt neues E
PUT	`http://library.com/books/78901`	Book	Book	Aktuali ein besteh Buch
DELETE	`http://library.com/books/78901`	-	-	Löscht Buch

ÜBUNG: STANDARDMETHODEN

Welches sind gültige Standardmethoden die REST mit HTTP nutzt?

- [ ] DELETE
- [ ] REMOVE
- [ ] GET
- [ ] PUT
- [ ] MODIFY
- [ ] POST

ÜBUNG: INTERPRETATION (1)

Gegeben sei der Aufruf

```
GET http://www.example.de/products/4711/producer
```

Welche Interpretation passt?

- ☐ Lies das Produkt mit der ID 4711
- ☐ Hole den Produzenten des Produkts mit der ID 4711
- ☐ Lies den Produzenten mit der ID 4711

ÜBUNG: INTERPRETATION (2)

Gegeben sei der Aufruf und Request-Body.

```
PUT http://www.example.de/customer/4711
```

```
{"id": "4711", "name": "Erna", "adresse": "Esplanade 10, 85055 Ingolstadt"}
```

Welche Interpretation passt?

- ☐ Erzeuge den Kunden mit der ID 4711
- ☐ Setze die ID des Kunden auf 4711
- ☐ Aktualisiere den Kunden mit der ID 4711 anhand der übergebenen Daten

UNTERSCHIEDLICHE REPRÄSENTATIONEN

- Es gibt heute viele gebräuchliche Repräsentationen für ein Fachobjekt - bspw. Text, XML, HTML, JSON.
- Über das Header-Attribut "Accept" kann der Client als MIME-Type angeben, welches Format er für das Ergebnis wünscht.
- Server unterstützen heute häufig nur das JSON-Format, es kann also durchaus sein, dass das gewünschte Format nicht geliefert wird!

Beispiel 1: Kunde mit ID 1234 im XML-Format

```
GET /Kunden/1234
Host: beispiel.de
Accept: application/xml
```

```
<Kunde>
    <nr>1234</nr>
    <name>Hermann Mann</name>
</Kunde>
```



Beispiel 2: Kunde mit ID 1234 im JSON-Format

```
GET /Kunden/1234
Host: beispiel.de
Accept: application/json
```

```
{
    "nr": 1234,
    "name": "Hermann Mann"
}
```

STATUSLOSE KOMMUNIKATION

HTTP ist ein zustandsloses Protokoll, d.h. jeder Aufruf wird mit den in ihm enthaltenen Daten und typischerweise den Daten aus anderen Quellen (Datenbank, Dateien, andere Server) beantwortet.

Vorteile

- Ausfallschutz durch Redundanz von gleichartigen Servern
- Höhere Skalierbarkeit durch den Einsatz gleichartiger Server-Instanzen in einem Server-Cluster

Nachteil

- Zusammengehörende Anfragen müssen durch zusätzliche Maßnahmen zusammen geführt werden - vgl. Sitzungsidentifikationen

ÜBUNG: REST-PRINZIPIEN

Was sind grundlegende Prinzipien von REST?

- [ ] Unterschiedliche Repräsentationen von Ressourcen
- [ ] Eindeutige Identifikation von Ressourcen über URIs
- [ ] Synchrone Kommunikation zwischen Clients
- [ ] Statuslose Kommunikation zwischen Client und Server
- [ ] Zwingend verschlüsselte Datenübertragung
- [ ] Verwendung von HTTP-Standardmethoden

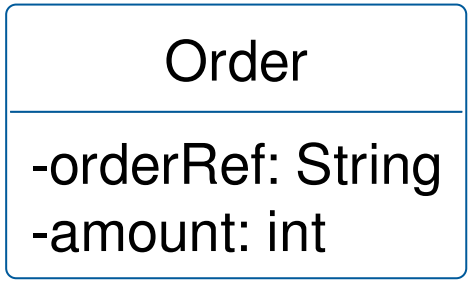
ÜBUNG: REST-URIS

Welche der folgenden URIs entsprechen den REST-Prinzipien?

- [ ] http://example.com/getCustomerById?id=1234
- [ ] http://example.com/orders/2024/03/776654
- [ ] http://example.com/products/4554
- [ ] http://example.com/customers/1234
- [ ] http://example.com/executeOrderProcess.php

ÜBUNG: REST-OPERATIONEN EINER SCHNITTSTELLE

Gegeben sei erneut die Bestellung. Beschreiben Sie was eine Beispiel URL, welche HTTP-Method, welcher Request-Payload und welcher Response-Payload für die Operationen (Abfragen eines Elements, Auflisten, Hinzufügen, Aktualisieren und Löschen) verwendet wird.





Beschreiben Sie die Schnittstelle:

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

QUELLEN

- S. Tilkov: REST und HTTP, Dpunkt Verlang, 2011
- S. Tilkov - REST for SOA (OOP-Konferenz 2008)
- Java 7 ist auch eine Insel, Kapitel 13, s. http://openbook.galileocomputing.de/java7/1507_13_001.html



9.2 REST-BEISPIEL

Anhand des Beispiels für eine Bestellungsverwaltung sollen die Grundprinzipien von REST in der Anwendung gezeigt werden. Das Beispiel teilt sich in Server und Client.

Server-Download:

Nutzen Sie das bereitgestellte Projekt-Archiv bestellungsverwaltung-server.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.



Test der Server-Installation:

Nach dem Download in den htdocs-Bereich eines Apache-Webservers platzieren und z.B. mit Postman auch ohne Client testen.

Aufruf zum Laden der Bestellungen:

GET http://localhost:80/bestellungsverwaltung-server-solution/bestellungen

Annahmen:

- der Webserver hört auf Port 80
- das ZIP-Archiv wurde unter `htdocs/bestellungsverwaltung-server-solution` entpackt



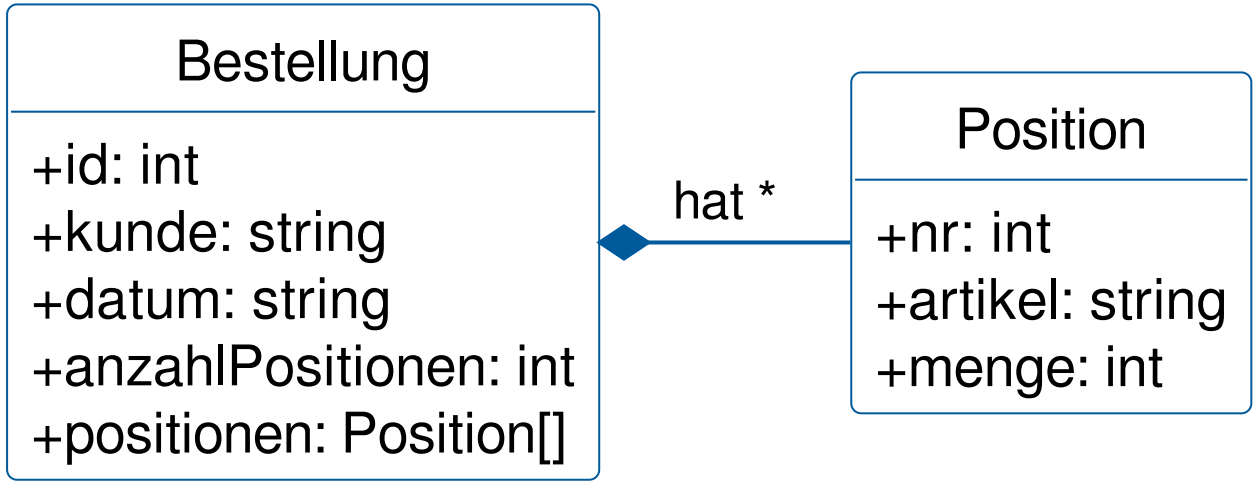
AJAX Client

Nutzen Sie das bereitgestellte Projekt-Archiv bestellungsverwaltung-client-simple.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

Die Funktionsweise des Clients sei hier kurz dargestellt:

- Zu Beginn wird eine leere Bestellliste angezeigt
- Nach Klick auf `lade Bestellungen` werden die Bestellungen per AJAX-Call vom Server geladen und mittels DOM-Manipulation angezeigt.
- Durch Anklicken einer Bestellung werden die zugehörigen Positionen geladen und ins DOM geschrieben.

Der Server nutzt folgendes Datenmodell



Anmerkungen:

1. Obige Assoziation zwischen Bestellung und Position ist redundant, da gleichzeitig das realisierende Attribut "positionen" vorhanden ist. Die Assoziation ist aus Gründen der Anschaulichkeit dennoch gezeigt.
2. Das Attribut `anzahlPositionen` ist redundant, wird aber vom Server geliefert, um die Anzahl der Positionen anzeigen zu können. Die Positionen selbst werden erst in einem zweiten Schritt geladen.



...und bietet folgende Schnittstelle für den AJAX-Client an:

BestellungenService
(1) + gibBestellungen(): Bestellung[] (2) + gibBestellung(id: int): Bestellung (3) + erzeugeBestellung(b: Bestellung): void (4) + aktualisiereBestellung(b: Bestellung): void (5) + loescheBestellung(id: int): void
(6) + ladePositionenZuBestellung(bestellungsId: int): Position[] (7) + erzeugePosition(bestellungsId: int, p: Position): void (8) + aktualisierePosition(bestellungsId: int, p: Position): void (9) + loeschePosition(bestellungsId: int, positionsNr: int): void

- Der erste Teil der Schnittstelle ((1) - (5)) bietet Grundfunktionen für die Ressource "Bestellung" an.
- Der zweite Teil ((6) - (9)) bietet Funktionen für die Sub-Ressource "Position" an.

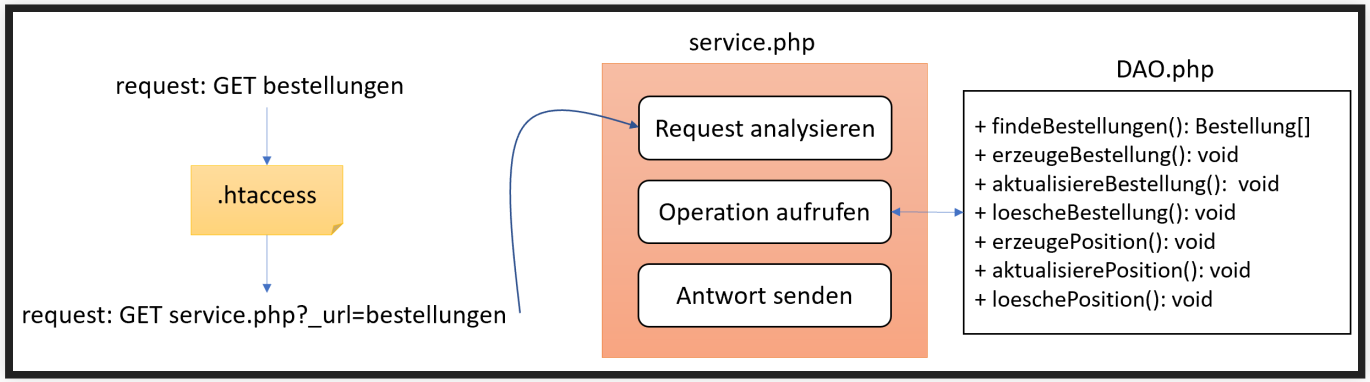
Gemäß der festgelegten Semantik der HTTP-Befehle lässt sich das Server-Interface bzgl. der Ressource *Bestellung* mit folgenden REST-Befehlen umsetzen:

Nr	Befehl	Pfad	request body	reply body	http reply code(s)
1	GET	bestellungen		Bestellung[]	200
2	GET	bestellung/{id}		Bestellung	200 id gültig 404 id ungültig
3	POST	bestellung	Bestellung		200
4	PUT	bestellung/{id}	Bestellung		200 id gültig 404 id ungültig
5	DELETE	bestellung/{id}			200 id gültig 404 id ungültig

Das Server-Interface bzgl. der Sub-Ressource *Position* lässt sich mit folgenden REST-Befehlen umsetzen:

Nr	Befehl	Pfad	request body	reply body	http reply code(s)
6	GET	bestellung/{id}/positionen		Position[]	200 id gültig 404 id ungültig
7	POST	bestellung/{id}/position	Position		200 id gültig 404 id ungültig
8	PUT	bestellung/{id}/position/{nr}	Position		200 id+nr gültig 404 id/nr ungültig

FUNKTIONSWEISE DES BESTELLUNGSSERVERS



Die Request-Abarbeitung funktioniert folgendermaßen:

1. Durch die "Rewrite-Rule" in der .htaccess-Datei wird der request an "service.php" umgeleitet, der ursprüngliche Pfad wird als Query-String erhalten.
2. service.php führt drei Schritte aus
    - Anhand Befehl, Pfad und Parametern wird die auszuführende Operation ermittelt.
    - Die Operation wird aufgerufen. Sie nutzt das DAO-Objekt, welches die Bestellungsdaten verwaltet.
    - Zuletzt wird der http code gesetzt und der reply-body wird ggf. befüllt.

Request-Analyse in service.php

```
$url = $_REQUEST['_url'];
$requestType = $_SERVER['REQUEST_METHOD'];
$body = file_get_contents('php://input');

try {
    if ($requestType === "OPTIONS") { ...
    } else if ($url === '/bestellungen') { ...
    } else if ($url === '/bestellung' && $requestType === "POST") { ...
    } else if (preg_match("/\/bestellung\/[0-9]*\/positionen/", $url) &&
        $requestType === 'GET') { ...
    }   ...
    else {
        throw new Exception('bad request');
    }
} catch (Exception $e) {
    badRequest($requestType, $url, $body);
}
```

- URL, HTTP-Befehl und HTTP-Body werden ermittelt.
- Falls ein OPTIONS-Befehl vorliegt, wird eine entsprechende Antwort gesendet.
- Die restlichen Fallunterscheidungen dienen der Erkennung der einzelnen Operationen. Hierzu werden Pfad und HTTP-Befehl untersucht. Die Pfad-Untersuchung gelingt mit regulären Ausdrücken, da die Pfade auch dynamische Anteile (z.B. Zahlen) enthalten können.

Die Request-Analyse wird am Beispiel der Erkennung des Aufrufs "GET /bestellung/4711/positionen" gezeigt.

```
} else if (preg_match("/\/bestellung\/[0-9]*\/positionen/", $url) &&
        $requestType === 'GET') {
    $parts = explode('/', $url);
    if (count($parts) !== 4) {
        throw new Exception('bad request');
    } else {
        $bestellungId = $parts[2];
        getPositionenZuBestellung($bestellungId);
    }
}
```

- explode() zerlegt den Pfad in einzelne Strings anhand des Pfadtrenners, hier: "", "bestellung", "4711", "positionen".
- Wenn es genau 4 Teilstrings gibt, wird die Bestellungs-Id als 3. Teilstring ermittelt und der Aufruf "getPositionenZuBestellung('4711')" durchgeführt.

Der Operationsdurchführung zur Berechnung einer Antwort wird am Beispiel der Funktion "getPositionenZuBestellung()" gezeigt.

```php
function getPositionenZuBestellung($bestellungId) {
    global $dao;

    $positionen = $dao->findePositionenZuBestellung($bestellungId);
    if ($positionen === NULL) {
        http_response_code(404);
    } else {
        echo json_encode($positionen);
        http_response_code(200);
    }
}
```

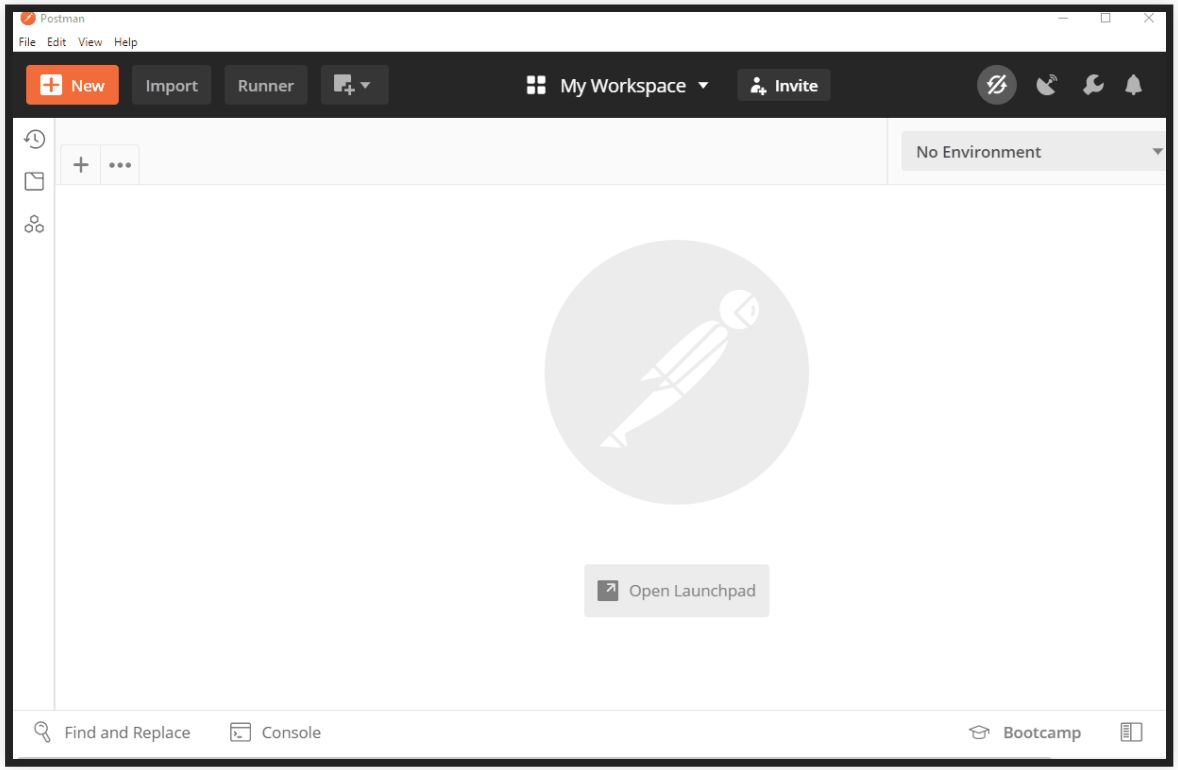
- Die Funktion nutzt das globale Data Access Object (DAO), welches die Bestellungsdaten über eine API zugänglich macht. Das DAO wird in einer Session als Singleton gehalten.
- Zunächst werden die Bestellpositionen zur Bestellung geholt.
- Existiert die Bestellung nicht, so wird NULL zurückgegeben, das Operationsergebnis ist 404 (not found).
- Andernfalls wird das Position-Array in JSON-Format umgewandelt und mit echo in den reply-body geschrieben.
- Zuletzt wird ein der http code für "Erfolg" gesetzt.

Weitere Anmerkungen:

- Reguläre Ausdrücke haben eine komplexe Syntax. Zur Entwicklung bzw. zum Test solcher Ausdrücke empfiehlt sich die Nutzung einer Seite wie Regex101.
- Cross-Origin Resource Sharing (CORS) ist ein Schutzmechanismus, der in Browsern verhindern soll, dass Skripte unerlaubt Daten mit anderen Servern austauschen:
    - Situation: Ein Skript wird von Host A geladen und sendet einen Request an Host B.
    - Der Browser erkennt CORS und sendet vor dem eigentlichen Request einen "preflight request", das ist eine OPTIONS-Anfrage, mit der quasi um Erlaubnis gebeten wird.
    - Sendet der Server in seiner Antwort entsprechende Header mit (z.B. "Access-Control-Allow-Origin: *"), so wird der Request gesendet, sonst unterbleibt er.

Entwicklung und Test des REST-Servers für Bestellungen

Für die schnelle und modulare Entwicklung ist es ratsam, den Client-Code nach dem Server-Code zu entwickeln. Als Ersatz-Client kann hier z.B. Postman genutzt werden. Folgende Animation zeigt, wie man mit Postman z.B. den Aufruf "GET bestellungen" testen kann.



9.3 VOR- UND NACHTEILE VON REST

Vorteile

- Vielfältig unterstützt in Bezug auf Programmiersprachen, Betriebssysteme etc.
- Skalierbarkeit
- Echte Web-Integration für Rechner-zu-Rechner-Kommunikation (JSON über http)
- Unterstützt unterschiedlichste Formate wie XML, JSON, Text, HTML, ...
- Durch Standard-Befehle (GET, ...) ergeben sich einfache Schnittstellen

Nachteile

- Keine Standardisierung im Bereich Design von REST-Schnittstellen
- REST oft falsch verstanden, z.B. Funktionen als Ressourcen
- Teilweise Schwierigkeiten bei der Abbildung von bestehender Funktionalität auf die in REST geforderte Ressourcen-Zentrierung



9.4 ÜBUNGEN

ÜBUNG: PROTOKOLL MIT REST

Die Übung für die Erfassung eines Gesprächsprotokolls aus dem PHP-Kapitel wird im Folgenden so erweitert, dass eine Anwendung unter Verwendung von AJAX, MVC und REST entsteht. Hierfür steht der finale Zustand aus dem AJAX-Kapitel bereits zur Verfügung.

Nutzen Sie das bereitgestellte Projekt-Archiv rest-protocol.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.



Anmerkungen

- In der Datei `api.php` finden Sie ähnliche Strukturen wie in der `service.php` aus dem REST-Beispiel
- Ergännzen Sie die Behandlung für den Pfad `/protocol` - einmal für das Auflisten und einmal für das Speichern
- Achten Sie darauf, dass sich im Vergleich zur AJAX-Lösung nichts am Controller geändert hat - Sie müssen weiterhin die `run`-Methode aufrufen und das `$data`-Objekt entsprechend anpassen
- Finalisieren Sie das Setting, indem Sie in der `index.html` das Laden und Speichern überarbeiten - hierfür existiert bereits eine vorbereitete Methode `createRequest`

```
function createRequest(method, url, data, callback) {
    var xmlhttp = new XMLHttpRequest();
    xmlhttp.onreadystatechange = function () {
        if (xmlhttp.readyState == 4 && xmlhttp.status == 200) {
            let result = JSON.parse(xmlhttp.responseText);
            callback(result);
        }
    };
    xmlhttp.open(method, url);
    if(method == "POST" || method == "PUT") {
        xmlhttp.setRequestHeader("Content-Type",
            "application/json;charset=UTF-8");
        xmlhttp.send(JSON.stringify(data));
    } else {
        xmlhttp.send();
    }
}
```

- Erzeugen einer Anfrage mit URL und HTTP-Methode
- Im Fall von POST und PUT wird JSON verwendet und das Datenobjekt versendet
- Behandlung der Antwort mithilfe der Callback-Methode

